

Property-based testing: Part 2

17-313: Foundations of Software Engineering

<https://cmu-313.github.io>

Michael Hilton and **Josh Sunshine**

Fall 2026

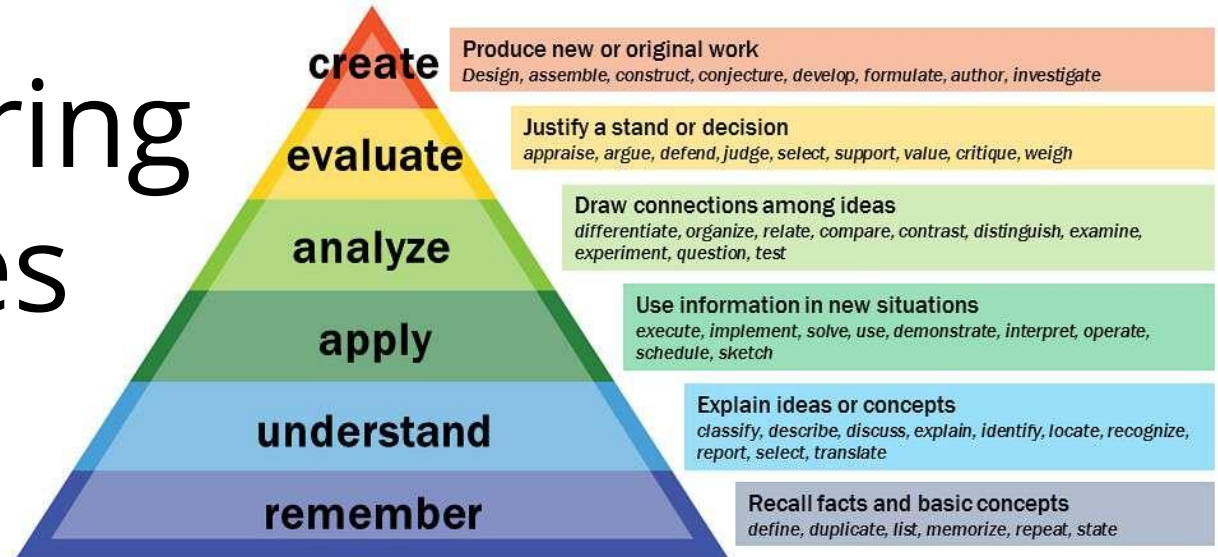
Administrivia

- Midterm on Thursday, October 8
 - Prior-year midterms released to you
 - Questions:
 - Half like prior year midterms
 - Half questions like problem sets and quizzes, *but **free response***
 - No cheat sheets
- Enjoy Fall Break!

Develop Engineering Judgement

Learn
Engineering
Principles

Bloom's Taxonomy



Vanderbilt University Center for Teaching

Conditional Properties

Conditional Properties

- **If** condition A is met, **then** property B must hold.
Mathematical logic: $A \Rightarrow B$
- Examples:
 - **Wordle**: If the guess word matches the secret word the output should be (G,G,G,G,G).
 - **Compilers**: If the program includes an invalid keyword, the parser should throw a parse error.
 - **NodeBB** converts post titles into URL-safe "slugs" (e.g., "Hello World!" becomes hello-world). If the input string contains at least one alphanumeric character then the generated slug must never be empty and must only contain a-z, 0-9, and -.

Conditional Properties Approach 1: Filtering

Most PBT frameworks allow you to "discard" inputs that don't meet your criteria. In fast-check you use the `fc.pre()` method.

- **How it works:** The generator creates a random withdrawal amount. If that amount is $\leq$ the balance, the test framework simply throws that case away and tries again with a new random number.
- **The Pro:** It's very easy to write.
- **The Con:** It can be inefficient. If your condition is very rare (e.g., "only when the amount is exactly 1,000,000.03"), the framework might exhaust its retry limit before finding enough valid cases.

Conditional Properties Approach 2: Targeted Generators

Instead of generating *any* number and throwing away the bad ones, you create a custom generator that *only* produces "invalid" inputs.

- **How it works:** You define a generator specifically for the error case: `amount = random_int(min=balance + 1)`.
- **The Pro:** Every single test run is a "hit." It's incredibly efficient and ensures you are testing the error handling thoroughly.
- **The Con:** It requires more setup code and a deeper understanding of your data constraints.

```
import * as fc from 'fast-check';
import { createHash } from 'crypto';

// 1. Define the internal data you want to vary
const rawDataArb = fc.record({
  id: fc.uuid(),
  amount: fc.integer({ min: 1, max: 10000 }),
  timestamp: fc.date().map(d => d.toISOString())
});
```

```
// 2. Create the "Targeted" Generator using .map()
// This ensures every single output has a perfectly valid
// integrity constraint.
const signedJsonArb = rawDataArb.map(data => {
  const payload = JSON.stringify(data);

  // Integrity Constraint: Generate the signature based on the payload
  const signature = createHash('sha256').update(payload).digest('hex');

  // Return the final "rare" input format
  return JSON.stringify({ payload, signature });
});
```

```
// 3. The Property Test
```

```
fc.assert(
```

```
fc.property(signedJsonArb, (jsonInput) => {
```

```
  // Every input here is guaranteed to have a valid signature
```

```
  const result = mySecureService.process(jsonInput);
```

```
  return result.status === 'ACCEPTED';
```

```
})
```

```
);
```

Conditional Properties Approach 3: Branching Properties

- **How it works:** The test itself has branches (e.g. if and else) and the property is defined across the branches.
- **The Pro:** This creates a "complete" specification of the function in one place.
- **The Con:** It can make the test code more complex and harder to debug if it fails.

In-class activity: Vibe analysis

Common Property: Differential

- Two implementations (or versions) **agree on outputs and errors**
 - Slow but trusted reference solution vs. optimized version
 - Old vs. new implementation after refactoring
 - Third-party library vs. your code

```
fc.assert(fc.property(fc.array(fc.integer()), xs => {  
  const a = doubleBogosort(xs);  
  const b = radixSort(xs);  
  return a.length === b.length && a.every((v,i)=> v === b[i]);  
}));
```

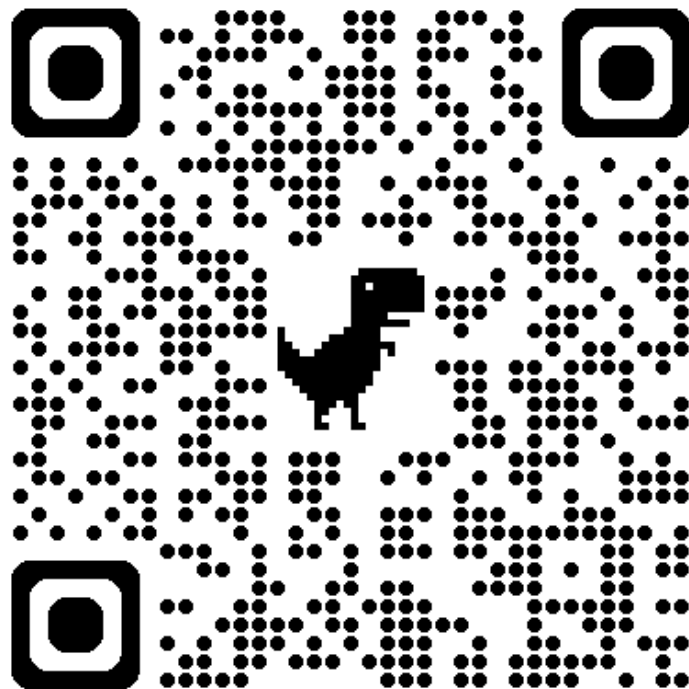
Making functions easy to test

- Side-effect free
- Determinism
- Well-defined input and output sets
- Small input and output sets

In-class activity

<https://tinyurl.com/313f26-vibe-analysis>

1. Use two different LLMs to create a testable TypeScript function
2. Find an example where the function's behavior is subtly different from between the two LLMs.
3. Write a fast-check test to determine where the behavior of the functions differ.
4. Put your example in the form.





Wishing all 17-313 Students a

**HAPPY
FALL BREAK!**

*Take a well-deserved rest!
See you soon.*



**CARNEGIE
MELLON
UNIVERSITY**